

Exercise 3: Sorting Customer Orders

1. Understand Sorting Algorithms

Bubble Sort

- Repeatedly compares adjacent elements.
- Swaps elements if they are in the wrong order.
- Simple but inefficient for large datasets.

Quick Sort

- Uses divide-and-conquer strategy.
- Selects a pivot and partitions the array.
- Very fast in practice.

2. Setup

```
class Order {  
    int orderId;  
    String customerName;  
    double totalPrice;  
  
    Order(int orderId, String customerName, double totalPrice) {  
        this.orderId = orderId;  
        this.customerName = customerName;  
        this.totalPrice = totalPrice;  
    }  
}
```

```
    public String toString() {  
        return "Order ID: " + orderId + " Customer: " + customerName + " Total Price:  
" + totalPrice;  
    }  
}
```

3. Implementation

```
class Sort {  
    public static void bubbleSort(Order[] orders) {  
        int len = orders.length;  
  
        for (int i = 0; i < len - 1; i++) {  
            for (int j = 0; j < len - i - 1; j++) {  
                if (orders[j].totalPrice > orders[j + 1].totalPrice) {  
                    Order temp = orders[j];  
                    orders[j] = orders[j + 1];  
                    orders[j + 1] = temp;  
                }  
            }  
        }  
    }  
}
```

```
    }  
  }  
}
```

```
public static void quickSort(Order[] orders, int low, int high) {  
    if (low < high) {  
        int pivotIndex = partition(orders, low, high);
```

```
        quickSort(orders, low, pivotIndex - 1);  
        quickSort(orders, pivotIndex + 1, high);  
    }  
}
```

```
public static int partition(Order[] orders, int low, int high) {  
    double pivot = orders[high].totalPrice;  
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {  
        if (orders[j].totalPrice < pivot) {  
            i++;
```

```
            Order temp = orders[i];  
            orders[i] = orders[j];  
            orders[j] = temp;  
        }  
    }
```

```
    Order temp = orders[i + 1];  
    orders[i + 1] = orders[high];  
    orders[high] = temp;
```

```
    return i + 1;  
}
```

```
public class SortingCustomers {  
    public static void main(String[] args) {  
        Order[] orders = {  
            new Order(101, "Alice", 5000),  
            new Order(102, "Bob", 3000),  
            new Order(103, "Charlie", 7000),  
            new Order(104, "David", 2000)  
        };
```

```
        System.out.println("Before Sorting:");  
        for (Order o : orders)  
            System.out.println(o);
```

```
Sort.quickSort(orders, 0, orders.length - 1);
```

```
System.out.println("After Sorting:");  
for (Order o : orders)  
    System.out.println(o);  
}  
}
```

4. Analysis

<i>Algorithm</i>	<i>Best</i>	<i>Worst</i>	<i>Average</i>
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Why Quick Sort is Preferred

- Much faster for large datasets.
- Uses divide-and-conquer strategy.
- Requires less memory than Merge Sort.
- Average performance is significantly better than Bubble Sort.